
SAE DEVELOPPEMENT

Compte rendu

0. Objectif du projet

L'objectif central du projet est de fournir une plateforme modulaire permettant d'héberger plusieurs jeux, accompagnée d'une interface utilisateur textuelle (TUI) et d'un mécanisme de menu extensible.

Dans cet environnement, un premier jeu complet est implémenté : le jeu de cartes "The War" (bataille), avec :

- une gestion complète d'un jeu de cartes abstrait (deck, joueurs, table) ;
- un système de distribution, tirage et comparaison de cartes ;
- une IA simple pour les joueurs non-humains ;
- la possibilité pour le joueur 1 de jouer en mode manuel ;
- des règles optionnelles (la règle "hyper two") ;
- un moteur d'affichage ASCII basé sur un système de cache pour accélérer les rendus ;
- un système de réglages configurables ;
- un mécanisme d'enregistrement automatique des jeux au démarrage.

Le projet vise donc simultanément :

0.1 La création d'un environnement de jeu générique

Grâce au module `menu.h`, n'importe quel jeu peut s'enregistrer automatiquement au démarrage via un objet statique.

Cela permet de construire une plateforme multi-jeux sans modifier le cœur du programme.

0.2 L'implémentation d'un jeu de cartes complet

Le jeu "The War" démontre un pipeline de jeu complet :

- création des joueurs ;
- distribution aléatoire des cartes ;
- boucle de rounds automatisée ;
- gestion des égalités ;
- transfert des cartes au gagnant ;
- système de victoire robuste.

0.3 Un moteur TUI modulable

Le composant TUI (Text User Interface) apporte :

- un cache de sections de fichiers texte ;
- un système de pré-chargement ;
- des fonctions d’affichage horizontales pour juxtaposer les cartes en ASCII art ;
- un mécanisme portable (via `EnableUTF8` / `EnableANSIColors`) pour exploiter des couleurs ANSI.

0.4 Une interaction utilisateur sûre

Grâce à `secure_cin.hpp`, les entrées utilisateur sont sécurisées, évitant :

- les erreurs de format ;
- les boucles infinies dues à `std::cin.fail()` ;
- les débordements de buffer.

En résumé, le projet constitue à la fois :

- un moteur de jeux générique extensible et
- une implémentation complète d’un premier jeu exploitant toutes ses fonctionnalités.

1. Architecture générale du projet

L’architecture repose sur quatre ensembles logiques principaux, interconnectés de manière cohérente et modulaire.

1.1 Le système menu (`menu.h`)

Ce système permet l’auto enregistrement au lancement du programme (à condition que les programmes aient été compilés ensemble).

Il est composé d’un conteneur global `std::vector<game>& getGameList()` qui expose un conteneur global, utilisé comme registre. Le type `game` (sutructure d’un jeu) encapsule un nom ainsi qu’une fonction de lancement (`std::function<void()>`).

Chaque jeu peut s’inscrire via `static register_Cardgame__the_war`
`registerGameInstance;`

1.2 Le moteur TUI (TUI-Engine—Prequel.h)

Le moteur est composé de 2 grandes unités :

1.2.1 TUI_Cache : cache de sections de fichiers

Cette classe gère :

- un `unordered_map<KeyStructure, vector<string>>`,
- la capacité de lire un fichier complet ou une section délimitée par un séparateur (par défaut `"==="`),
- l’effacement d’entrées à granularité fine.

Le cache optimise les performances en évitant :

- les relectures de fichiers ASCII volumineux ;
- la recomposition des cartes ASCII à chaque affichage.

1.2.2 TUI_Render : interface d’affichage

Construit à partir du cache, il permet :

- `preload` : précharger des fichiers
- `display` : afficher un fichier ou une section ;
- `horizontal_display` : juxtaposer des sections pour aligner les cartes ASCII ;
- `clearScreen` : rafraîchir l’affichage.

TUI_Core instancie ces deux blocs et expose un membre public : `TUI_Render render` ; permettant aux jeux d’utiliser directement les fonctionnalités TUI.

2.3 Le noyau des jeux de cartes

Ce module définit trois types majeurs :

2.3.1 Deck

- contient soit 32 cartes, soit 52 cartes selon le constructeur ;
- stocke les cartes dans un `unordered_map` ;
- supporte :
 - le tirage par clé,
 - le tirage aléatoire,
 - l’égalité stricte via `operator==`,
 - la conversion clé → carte.

2.3.2 Player

- Chaque joueur possède :
- un `unordered_map<key, Card>` pour l’accès rapide,
- un `vector<key>` pour l’ordre d’affichage,
- les primitives :
 - ajouter une carte ;
 - jouer une carte spécifique ;
 - jouer la première carte (FIFO) ;
 - récupérer la taille de sa main.

2.3.3 Table

- joue le rôle de “pot” commun ;
- stocke les cartes jouées dans l’ordre ;
- autorise la récupération par clé.

Ce module fournit l'abstraction générale nécessaire à n'importe quel jeu de cartes.

2.4 Le jeu "The War"

L'implémentation du jeu regroupe :

2.4.1 Des variables globales de configuration

- nombre de joueurs ;
- mode manuel ;
- règle spéciale.

Toutes encapsulées dans des getters/setters pour éviter les modifications accidentelles.

2.4.2 Le pipeline de jeu

- initialisation du moteur TUI ;
- distribution aléatoire
- boucle de rounds :
 - sélection d'une carte par joueur ;
 - affichage ASCII ;
 - évaluation du gagnant (avec ou sans règle spéciale) ;
 - transfert des cartes ;
 - vérification de la victoire.

2.4.3 Le menu spécifique au jeu

Inclut :

- Play
- Settings
- Credits
- Exit

Chaque section utilise le moteur TUI pour afficher ASCII art et instructions.

2. Choix de conception adoptée

2.1 Utilisation d'un registre global pour les jeux

Le choix d'exposer un conteneur global via `getGameList()` et d'y inscrire les jeux par des objets statiques représente une conception modulaire et extensible.

2.2 Moteur TUI basé sur un cache

L'affichage ASCII, notamment pour un jeu de cartes est coûteux (fichiers volumineux, nombreux accès).

Un cache évite la relecture systématique et améliore grandement les performances.

Cela permet une meilleure rapidité d'affichage, une indépendance complète entre contenu ASCII et logique de jeu ainsi qu'une séparation claire des responsabilités (cache / rendu).

2.3 Représentation des cartes par clé texte

Utiliser "14S" ou "7H" comme clé est un choix simple et robuste.

Avantages :

- Unique, compact, directement indexable.
- Compatible avec un `unordered_map` (hash efficace).
- Convertibilité immédiate pour le rendu ASCII (nom de section = clé).

2.4 Gestion de la main du joueur via map + vector

Cette double structure combine :

- un accès rapide (`map`) -> utile pour les choix de carte ;
- un ordre logique (`vector`) -> utile pour l'affichage.

Ce qui permet un maintien simple

2.5 Boucle de jeu simple et sécurisée

Les règles du jeu sont réduites à leur essence :

- chaque joueur joue une carte ;
- la plus forte gagne ;
- les égalités sont résolues au hasard

2.6 Système de paramètres basé sur des variables statiques

Les paramètres globaux (ex : nombre de joueurs, règles spéciales) sont gérés par :

- une variable statique dans une fonction ;
- des booleans globaux dans le fichier de jeu.

Cela rend les paramètres plus simple à utiliser et les rends persistent entre les appels.

2.7 Fonction `secure_cin` pour gérer les entrées

Cette abstraction protège des erreurs communes à `std::cin`, notamment:

- la boucle d'erreur ;
- le leftover de buffer.

Ainsi les interactions utilisateurs sont plus robustes

3. Difficultés rencontrées et solutions trouvées

Durant ce projet, j'ai rencontré très vite plusieurs difficultés. La première étant l'implémentation des jeux. En effet, après la création du début de mon menu, je me suis très vite rendu compte que hard coder les fonctions des jeux dans le menu n'était pas pratique voir pouvait être source d'erreur. C'est la raison pour laquelle j'ai développé mon système d'auto enregistrement des jeux. Une autre difficulté que j'ai rencontrée est l'affichage. En effet afficher juste du texte était très peu esthétique, et mettre l'ASCII art dans le code allait le rendre très vite incompréhensible et inmaintenable. Ce n'est que plus tard, après avoir créé mon fichier texte qui contient les cartes que je me suis rendu compte que le nombre d'accès et le besoin d'une lecture rapide n'était pas compatible avec une lecture fichier à chaque appel d'où la création du cache. Une autre difficulté que j'ai rencontrée avant la création du cache, lorsque je testais le menu était les entrées utilisateur. Après avoir accidentellement entré une mauvaise valeur et que cela m'ait provoqué une boucle infinie, j'ai décidé de créer une couche d'abstraction (`secure_cin`) pour éviter ce genre d'erreurs, d'autant plus que toutes les interactions se font via des entrées utilisateurs. Enfin, un dernier point qui me posait problème était le fait que le code d'échappement AINSI était désactivé par défaut et que l'utf-8 était mal utilisé, d'où la création de `Set_windows_settings`.